GPS Spoofing Detection for PX4-Based Unmanned Aerial Vehicles

A Software-Only Approach Using Sensor Fusion and Machine Learning

Nikhil Sarwara

SIT724 / SIT746 Research Project
May 2026

Contents

1 Introduction **3**
1.1 Background and Motivation . . . 3
1.2 Spoofing versus Jamming: A Critical Distinction . . . 3
1.3 PX4 Architecture and the EKF2 Vulnerability . . . 3
1.4 Research Questions . . . 4

2 Literature Context **4**
2.1 Related Work . . . 4

3 Data Capture and Simulation Environment **5**
3.1 The Docker–Gazebo–PX4 SITL Stack . . . 5
3.2 Data Sources and Telemetry Streams . . . 5
3.3 Capture Procedures . . . 5
3.4 Dataset Scope . . . 6

4 Preprocessing Pipeline **6**
4.1 Pipeline Overview . . . 6
4.2 Sliding Window Mathematics . . . 6
4.3 Heuristic Labelling Rules . . . 6
4.3.1 Heuristic 3: GPS–IMU Consistency Score ($\Delta\Phi$) . . . 7
4.4 Feature Space . . . 7

5 Model Architectures **7**
5.1 Random Forest Classifier . . . 7
5.1.1 Input Flattening . . . 7
5.2 One-Dimensional Convolutional Neural Network . . . 7
5.2.1 Training Configuration . . . 8

6 Results and Performance Metrics **8**
6.1 Dataset Summary . . . 8
6.2 Classification Results . . . 8
6.3 The CNN Paradox: Interpreting the False Positive Rate . . . 9
6.4 Live Inference Results . . . 9

7 Proposed Mitigation Strategies **10**
7.1 Post-Detection Response Protocol 10
7.2 GPS Trust Index Design 10

8 System Limitations and Future Work **11**
8.1 Latency Analysis and Theoretical Maximum Allowed Latency 11
8.2 Breaking Point and Sensitivity Analysis 11
8.3 Limitations Affecting Generalisability 11

9 Conclusion **12**

1 Introduction

1.1 Background and Motivation

The proliferation of unmanned aerial vehicles (UAVs) across civilian infrastructure inspection, agricultural monitoring, emergency response, and military reconnaissance has elevated concerns regarding their vulnerability to Global Positioning System (GPS) spoofing attacks. In such attacks, adversaries transmit counterfeit GPS signals—synthesised with correct timing and structure but manipulated coordinates—to usurp legitimate satellite navigation. Unlike jamming, which merely attenuates or disrupts the GPS signal causing loss of lock, spoofing deceives the receiver into accepting falsified position, velocity, and time (PVT) estimates. The PX4 flight stack, which powers a substantial fraction of commercial and research drones, relies heavily on GPS as a primary input to its Extended Kalman Filter (EKF2) for state estimation. A successful spoofing attack can therefore cascade through the entire control pipeline, producing catastrophic trajectory deviations without triggering conventional failure modes.

Existing countermeasures frequently depend on specialised hardware—multi-antenna arrays, inertial navigation systems (INS) with tight coupled integration, or military-grade encrypted signals (M-code, SAASM)—that are incompatible with the SWaP-constrained (Size, Weight, and Power) platforms that dominate the commercial UAV market. A software-only detection mechanism that leverages sensors already present on standard PX4 autopilots represents a compelling alternative, provided it can achieve sufficient discriminative power in real time.

1.2 Spoofing versus Jamming: A Critical Distinction

The cybersecurity literature often conflates jamming and spoofing, yet their attack mechanics and detectability differ markedly:

- **Jamming** broadcasts high-power noise in the L1 band, raising the noise floor until the receiver loses carrier-phase lock. Detection is straightforward: the receiver reports a rapid drop in carrier-to-noise ratio and an explicit loss-of-fix status.
- **Spoofing** transmits structurally valid GPS signals with manipulated navigation data. Because the counterfeit signals comply with the GPS signal-in-space interface specification (IS-GPS-200), the victim receiver synchronises to the attacker-generated signals, computes an incorrect but internally consistent PVT solution, and reports a healthy fix. No carrier-to-noise anomaly is observed; the attack is silent until position deviation becomes physically obvious.

This distinction is operationally significant: jamming denies service, whereas spoofing corrupts service. A drone under spoofing continues to receive GPS updates and believes it is correctly localised, making software-level detection the only viable line of defence on cost-constrained platforms.

1.3 PX4 Architecture and the EKF2 Vulnerability

The PX4 autopilot employs EKF2, a 24-state Extended Kalman Filter that fuses GPS, barometer, magnetometer, and IMU data to estimate position, velocity, attitude, and sensor biases. Under normal operation, the EKF2 weights GPS position updates according to their reported accuracy (eph, epv). When spoofed GPS data is injected:

1. The EKF2 accepts the falsified position as a high-confidence observation because the spoofed signal reports low HDOP.
2. The filter state is dragged toward the attacker-controlled coordinates.

3. IMU integration, which reflects the drone's true motion, accumulates increasing innovation residuals relative to the corrupted GPS input.
4. If the innovation exceeds the EKF2's outlier gate, the filter may reject GPS entirely and revert to dead reckoning—but only if the discrepancy is severe enough and sustained long enough.

The latency of this natural rejection mechanism is unacceptably long for safety-critical flight. A dedicated anomaly detector that cross-validates GPS against IMU-derived motion estimates can flag spoofing before the EKF2 state is significantly corrupted.

1.4 Research Questions

This study is organised around three interdependent research questions:

RQ1: What are the primary vulnerabilities of GPS in autonomous mobile robots to spoofing attacks?

RQ2: How can sensor fusion techniques effectively detect GPS spoofing in real time?

RQ3: How can a dynamic GPS Trust Index be designed to quantify the reliability of GPS data?

To evaluate RQ2 and RQ3, the following hypotheses guide the empirical analysis:

H1: GPS spoofing introduces measurable inconsistencies between GPS-reported position and inertial measurement unit (IMU) data.

H2: Heuristic-based labelling can generate sufficiently accurate training data without manual annotation.

H3: Machine learning models (Random Forest and 1D CNN) can distinguish normal flight windows from spoofed windows.

H4: The live inference pipeline can operate with latency suitable for real-time detection.

2 Literature Context

2.1 Related Work

GPS spoofing detection has been approached through cryptographic, physical-layer, and sensor-fusion paradigms. Kerns et al. (2014) demonstrated civil GPS spoofing using low-cost software-defined radios, establishing that the threat is neither theoretical nor prohibitively expensive. Humphreys (2012) showed that civil UAVs could be captured via spoofing within minutes. Countermeasures include:

- **Cryptographic authentication:** Chimera, Galileo OSNMA, and GPS M-code use encrypted signatures to validate navigation data. These are unavailable on civil receivers.
- **Angle-of-arrival (AOA) discrimination:** Multi-antenna arrays detect signal direction mismatches. Effective but heavy.
- **Inertial cross-check:** Broumandan et al. (2020) and others have demonstrated that IMU-GPS inconsistency is a robust spoofing indicator, though most prior work assumes high-grade IMUs or post-flight analysis.

The present work differs by targeting real-time detection on standard PX4 telemetry with no hardware augmentation, bridging a gap between academic demonstrations and deployable software-only solutions.

3 Data Capture and Simulation Environment

3.1 The Docker–Gazebo–PX4 SITL Stack

To ensure reproducible, scalable experimentation, the simulation environment is containerised using Docker. The stack comprises:

1. **PX4 SITL (Software-In-The-Loop):** The PX4 flight control firmware runs natively on the host, communicating with the simulated vehicle via MAVLink over UDP.
2. **Gazebo Harmonic:** The physics engine simulates a 3 kg quadcopter with realistic aerodynamics, rotor thrust curves, and sensor noise models.
3. **QGroundControl / MAVSDK:** Ground control software initiates mission waypoints, allowing scripted injection of spoofing waypoints mid-flight.
4. **GPS Monitor:** A Python telemetry logger connects via pymavlink to the MAVLink UDP bridge, recording GLOBAL_POSITION_INT, HIGHRES_IMU, ATTITUDE, and GPS_STATUS at 10 Hz.

This stack enables controlled, repeatable attack scenarios without risking physical hardware. All flight logs are timestamped and archived for reproducibility.

3.2 Data Sources and Telemetry Streams

Table 1 summarises the telemetry channels, message frequencies, units, and analytical roles.

Table 1: Telemetry sources, frequencies, units, and analytical roles.

Source	MAVLink Msg	Freq.	Units	Purpose
PX4 SITL + Gazebo	GLOBAL_POSITION_INT	10 Hz	deg, m	Position, velocity, heading
	HIGHRES_IMU	10 Hz	m/s^2, rad/s	Acceleration, gyro rates
	ATTITUDE	10 Hz	rad	Roll, pitch, yaw
	GPS_STATUS	1 Hz	dimless	Satellite count, fix type
GPS Monitor	Raw CSV logs	10 Hz	mixed	Telemetry collection
ML Pipeline	Processed CSV	windowed	mixed	Cleaned, labelled, scaled
Live Inference	Timestamped CSV	10 Hz	mixed	Real-time detection outputs

3.3 Capture Procedures

Each experimental flight follows a scripted mission protocol:

1. Launch PX4 SITL and Gazebo; arm and take off to 20 m altitude.
2. Execute a rectangular survey pattern at 5 m/s.
3. At a random waypoint, inject spoofing by commanding a setpoint offset of 50 m north and 30 m east.
4. Maintain spoofed trajectory for 30 s, then either release (return to true GPS) or continue spoofing until mission end.
5. Log all telemetry; post-process to extract 10 Hz aligned time series.

3.4 Dataset Scope

The present analysis is based on 8 simulated flight runs, generating 4,207 telemetry records at 10 Hz. After preprocessing and windowing, this yields 76 windows (53 training, 11 validation, 12 test). While modest in scale, the dataset provides proof-of-concept validation under controlled conditions. Future work will expand to physical hardware and diverse attack profiles.

4 Preprocessing Pipeline

4.1 Pipeline Overview

Raw telemetry CSVs undergo a five-stage preprocessing pipeline:

1. **Cleaning:** Parse MAVLink messages, align timestamps, convert GPS to local Cartesian coordinates, and remove duplicates.
2. **Feature Engineering:** Compute velocity magnitude, heading delta, distance-to-origin, and GPS–IMU consistency scores.
3. **Labelling:** Apply eight heuristic rules to assign anomaly flags per row.
4. **Windowing:** Aggregate rows into 30-message sliding windows with 15-message stride.
5. **Splitting:** Partition windows into 70%/15%/15% train/val/test with flight-level isolation to prevent data leakage.

4.2 Sliding Window Mathematics

Let m_t denote a telemetry record at discrete time index t, and let $l_t \in \{0, 1\}$ be its heuristic label (0 = normal, 1 = spoofed). A sliding window of length $n = 30$ is defined as:

$$W_t = \{m_{t-n+1}, \ldots, m_t\} \tag{1}$$

The label for window W_t is determined by majority vote:

$$L(W_t) = \text{mode}(l_{t-n+1}, \ldots, l_t) = \begin{cases} 1 & \text{if } \sum_{i=t-n+1}^{t} l_i > \frac{n}{2} \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

With a stride of $s = 15$, consecutive windows overlap by 50%, ensuring temporal continuity in the training signal. At 10 Hz, each window spans approximately 3 s of flight time.

4.3 Heuristic Labelling Rules

Eight physics-based heuristics assign anomaly labels without manual annotation. Table 2 defines four representative rules in mathematical form.

Table 2: Selected heuristic labelling rules with mathematical definitions.

ID	Name	Mathematical Condition
H1	Position Jump	$\|\mathbf{P}_t - \mathbf{P}_{t-1}\| > \tau_{\text{pos}} = 5$ m
H2	Velocity Spike	$\|\mathbf{V}_t\| - \|\mathbf{V}_{t-1}\| > \tau_{\text{vel}} = 5$ m/s
H3	**GPS–IMU Consistency**	$\Delta\Phi = \left\| \int_{t_1}^{t_2} \mathbf{a}_{\text{imu}}\, dt - (\mathbf{P}_{\text{gps},t_2} - \mathbf{P}_{\text{gps},t_1}) \right\| > \tau$
H4	Altitude Discontinuity	$\|h_t - h_{t-1}\| > \tau_{\text{alt}} = 3$ m

4.3.1 Heuristic 3: GPS–IMU Consistency Score ($\Delta\Phi$)

The GPS–IMU consistency score is the central discriminator in this pipeline. The discrepancy between GPS-reported displacement and IMU-integrated displacement over interval $[t_1, t_2]$ is:

$$\Delta\Phi = \left\| \int_{t_1}^{t_2} \mathbf{a}_{\text{imu}}(t)\, dt - (\mathbf{P}_{\text{gps},t_2} - \mathbf{P}_{\text{gps},t_1}) \right\| \tag{3}$$

where $\mathbf{a}_{\text{imu}}(t)$ is the IMU-reported specific force (acceleration corrected for gravity), and $\mathbf{P}_{\text{gps},t}$ is the GPS-reported local Cartesian position. If $\Delta\Phi > \tau$ (empirically set to 8 m for the present drone dynamics), the interval is flagged as spoofed.

This heuristic is particularly robust because natural disturbances such as wind gusts affect *both* GPS and IMU—the former via ground-track deviation, the latter via aerodynamic perturbation—whereas spoofing corrupts GPS alone while IMU remains faithful to true motion.

The remaining heuristics (H5–H8) include: heading reversal without corresponding IMU yaw rate; stale repeat coordinates; geometric impossibility (position change incompatible with velocity); and compound anomaly (multiple simultaneous inconsistencies).

4.4 Feature Space

Each telemetry record contains 13 scalar features: local position (x, y, z), velocity (v_x, v_y, v_z), speed, heading, heading delta, distance to origin, acceleration magnitude, and the GPS–IMU consistency score. A 30-message window therefore produces a 30×13 feature matrix, which is flattened to a 390-dimensional vector for the Random Forest, or presented as a 13×30 tensor for the 1D CNN.

5 Model Architectures

5.1 Random Forest Classifier

The Random Forest (RF) is an ensemble of $B = 100$ decision trees trained via bootstrap aggregating (bagging). Each tree partitions the feature space using Gini impurity:

$$G = 1 - \sum_{k=1}^{K} p_k^2 \tag{4}$$

where p_k is the proportion of class k samples in a node. For this binary task ($K = 2$), trees are grown to full depth (unpruned) to maximise variance reduction.

5.1.1 Input Flattening

The window tensor $\mathbf{X} \in \mathbb{R}^{30\times13}$ is flattened row-major into a vector $\mathbf{x} \in \mathbb{R}^{390}$:

$$\mathbf{x} = \text{vec}(\mathbf{X}^\top) = [x_{1,1}, x_{1,2}, \ldots, x_{30,13}]^\top \tag{5}$$

This preserves the temporal ordering implicitly: the RF learns which feature-time combinations are discriminative without explicit convolutional operations.

5.2 One-Dimensional Convolutional Neural Network

The 1D CNN treats each window as a multivariate time series with 13 channels and 30 time steps. The architecture is detailed in Table 3.

Table 3: 1D CNN architecture for window classification.

Layer	Type	Output Shape	Parameters	Activation
Input	–	13×30	–	–
L1	Conv1D	32×30	kernel=3, stride=1, padding=1	ReLU
L2	Conv1D	64×30	kernel=3, stride=1, padding=1	ReLU
L3	AdaptiveAvgPool1D	64×1	–	–
L4	Flatten	64	–	–
L5	Dense	32	–	ReLU
L6	Dropout	32	$p = 0.3$	–
L7	Dense	1	–	Sigmoid

5.2.1 Training Configuration

The model is trained with the Adam optimiser ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) at a learning rate of 10^{-3}. Binary cross-entropy loss is minimised over 50 epochs with early stopping triggered when validation loss plateaus for 10 consecutive epochs. The training set contains 53 windows; validation contains 11.

6 Results and Performance Metrics

6.1 Dataset Summary

Table 4 presents the complete dataset characteristics.

Table 4: Dataset summary.

Metric	Value
Total flight runs	8
Total telemetry records	4,207
Sampling rate	10 Hz
Normal windows	8
Spoofed windows	68
Total windows generated	76
Window size	30 messages ($\sim$3 s)
Training windows	53
Validation windows	11
Test windows	12

The severe class imbalance (68 spoofed vs. 8 normal windows) reflects the experimental design: flights were predominantly spoofed to maximise attack data. Future datasets will include more normal flights for balanced evaluation.

6.2 Classification Results

Table 5 reports binary classification metrics on the held-out test set (12 windows).

Table 5: Model performance comparison on the test set.

Model	**Accuracy**	**Precision**	**Recall**	**F1**
Random Forest	1.0000	1.0000	1.0000	1.0000
1D CNN	0.8333	0.8333	1.0000	0.9091

Table 6: Confusion matrices for both models.

(a) Random Forest

		Predicted	
		Normal	Spoofed
Actual	Normal	2 (TN)	0 (FP)
	Spoofed	0 (FN)	10 (TP)

(b) 1D CNN

		Predicted	
		Normal	Spoofed
Actual	Normal	0 (TN)	2 (FP)
	Spoofed	0 (FN)	10 (TP)

6.3 The CNN Paradox: Interpreting the False Positive Rate

A striking finding emerges from Table 6: the 1D CNN achieves perfect recall (all 10 spoofed windows detected) but misclassifies both normal test windows as spoofed, yielding a 100% false positive rate on the limited normal sample. The Random Forest, by contrast, achieves perfect classification across all test instances.

Several factors explain this divergence:

1. **Dataset scale and class imbalance.** With only 53 training windows—of which the vast majority are spoofed—the CNN's convolutional kernels may learn "always predict spoofed" as a locally optimal strategy, since the prior probability of spoofing is overwhelming. The RF, with its ensemble averaging and explicit Gini splitting, appears more robust to this imbalance.

2. **Feature representation.** The RF operates on a flattened 390-dimensional vector where each scalar is directly interpretable. The CNN, by contrast, learns abstract temporal filters whose activations may overfit to spoofing-specific transients present in the training data.

3. **Overfitting risk.** The CNN has approximately 8,000 trainable parameters (Table 3) relative to 53 training samples—a parameter-to-sample ratio of $\sim$150:1, far exceeding conventional overfitting thresholds. Dropout ($p = 0.3$) and early stopping mitigate but do not eliminate this risk.

4. **Normal-flight diversity.** The 2 normal test windows likely represent flight dynamics not well represented in the training distribution (e.g., aggressive manoeuvres that the CNN associates with spoofing transients).

This "CNN Paradox" underscores a critical methodological caution: high accuracy in isolation is insufficient; the confusion structure must be examined, and class-imbalanced datasets demand stratified evaluation. Future work will employ class-weighted loss, oversampling, and larger normal-flight cohorts to address this limitation.

6.4 Live Inference Results

Recent live inference logs (`live_20260510_050706.csv`) demonstrate the detector operating on streaming telemetry. The log contains 1,164 records spanning approximately 109 seconds (15:05:17 to 15:07:06), during which the detector flagged 47 windows as anomalous.

During spoofed flight segments, the GPS position stream deviated from the IMU-reported motion, producing a measurable inconsistency score. The detector flagged these windows as anomalous, while normal flight segments with aggressive manoeuvres (where both GPS and IMU agreed) were correctly classified as normal. Quantitatively, the test set contained 10 spoofed windows and 2 normal windows; the Random Forest correctly classified all 12, while the CNN correctly identified all 10 spoofed windows but misclassified both normal windows as spoofed.

7 Proposed Mitigation Strategies

7.1 Post-Detection Response Protocol

Detection is only the first phase of a resilient navigation architecture. Upon anomaly detection, the drone should execute a structured response protocol rather than merely logging the event. The following strategies are proposed, ordered by escalating severity:

1. **GPS Trust Index (GTI) Degradation.** Maintain a dynamic scalar $G \in [0, 1]$ representing GPS confidence. Upon detection, attenuate G using an exponential decay:

$$G_{t+1} = G_t \cdot e^{-\lambda \cdot D_t} \tag{6}$$

   where $D_t \in \{0, 1\}$ is the detector output and λ is a time constant (empirically set to 0.5). The EKF2 uses G as a multiplicative weight on GPS observation covariance.

2. **Sensor Re-weighting.** As G falls below 0.3, the EKF2 should increase the relative weighting of barometric altitude, magnetometer heading, and optical flow (if available), reducing dependence on GPS position fixes.

3. **Hold Mode Activation.** If $G < 0.1$ for more than 5 s, the flight controller transitions to HOLD mode: maintain current position using IMU dead reckoning, reduce velocity to zero, and await operator intervention. This prevents the drone from flying into spoofed waypoints.

4. **Emergency RTL (Return-to-Launch).** If GPS is fully distrusted and the drone retains sufficient battery margin, execute an RTL using the last trusted home coordinates and IMU-compassed heading. This is a last-resort manoeuvre; accuracy degrades with distance due to IMU drift.

5. **Alert Broadcast.** Transmit a MAVLink STATUSTEXT message to the ground station with anomaly classification and recommended action, enabling human-in-the-loop decision making.

7.2 GPS Trust Index Design

The GTI is a direct response to **RQ3**. Rather than a binary "trust / distrust" flag, the GTI provides a continuous measure of GPS reliability:

$$G_t = \alpha \cdot G_{t-1} + (1 - \alpha) \cdot \left(1 - \text{sigmoid}(k \cdot \Delta\Phi_t - \theta)\right) \tag{7}$$

where $\alpha = 0.95$ provides temporal smoothing, k scales the consistency score, and θ is a bias term. The sigmoid maps the physical inconsistency $\Delta\Phi$ to a probabilistic trust score. This design allows graceful degradation rather than abrupt GPS rejection.

8 System Limitations and Future Work

8.1 Latency Analysis and Theoretical Maximum Allowed Latency

The 30-message window introduces a detection delay of approximately 3 s at 10 Hz. For safety-critical operations, this latency must be compared against the time available to react. Consider a drone flying at $v = 15$ m/s toward an obstacle:

$$t_{\text{react}} = \frac{d_{\text{safe}}}{v} \tag{8}$$

If the minimum safe braking distance is $d_{\text{safe}} = 30$ m, the available reaction time is:

$$t_{\text{react}} = \frac{30 \text{ m}}{15 \text{ m/s}} = 2 \text{ s} \tag{9}$$

The 3 s window delay therefore exceeds the available reaction time for this scenario, highlighting a critical limitation. Mitigations include:

- Reducing window size to 20 messages (2 s) at the cost of feature stability.
- Implementing a two-tier detector: a fast heuristic pre-filter (single-message latency, higher false positive rate) cascaded with the ML model.
- Increasing MAVLink polling frequency to 20 Hz or 50 Hz where bandwidth permits.

8.2 Breaking Point and Sensitivity Analysis

The current dataset employs abrupt spoofing with large position offsets (50 m). The minimum spoofing intensity detectable by the system—the "breaking point"—has not been characterised. Systematic sensitivity analysis is planned to quantify detection degradation for gradual drift attacks with velocity bias < 0.5 m/s. Such attacks are particularly insidious because they may not trigger the position-jump heuristic (H1) while slowly corrupting the EKF2 state.

8.3 Limitations Affecting Generalisability

1. **Simulation-only data.** Real-world RF environments, sensor hardware noise, and multi-path effects are not represented.
2. **Limited spoofing diversity.** Gradual drift, intermittent spoofing, and multi-sensor attacks are underrepresented.
3. **Heuristic label bias.** The eight heuristic rules may misclassify edge cases or dominate labels unevenly.
4. **Incomplete IMU integration.** Full IMU features (vibration, clipping counters) are not yet incorporated into the ML training input.
5. **Potential overfitting.** High accuracy on a limited dataset may indicate overfitting rather than generalisable performance.
6. **Latency measurement.** Formal real-time latency benchmarks have not yet been conducted.
7. **Breaking point unknown.** The minimum spoofing intensity for reliable detection has not been quantified.
8. **No physical validation.** The system has not been tested on real PX4 hardware or in live RF conditions.

9 Conclusion

This study demonstrates that a software-only GPS spoofing detection system, leveraging standard PX4 telemetry and cross-sensor consistency checks, can reliably identify spoofing behaviour in controlled simulation environments. The three research questions are addressed as follows:

- **RQ1:** The primary vulnerability is the EKF2's reliance on GPS as a high-weight observation; spoofed signals report healthy fix quality, causing the filter to accept corrupted data without immediate rejection.
- **RQ2:** Sensor fusion via the GPS–IMU consistency score ($\Delta\Phi$) provides a discriminative signal that distinguishes spoofing from natural disturbances, enabling both heuristic and ML-based real-time detection.
- **RQ3:** The proposed GPS Trust Index (GTI) offers a continuous, probabilistic measure of GPS reliability that enables graduated response protocols rather than binary trust/distrust decisions.

The Random Forest achieved perfect test-set classification, while the 1D CNN exhibited perfect recall but 100% false positive rate on the limited normal sample—a finding that underscores the need for larger, balanced datasets and careful overfitting analysis. The 3 s window latency exceeds theoretical safe-reaction time for high-speed scenarios, motivating future work on faster detection architectures.

Specific contributions include: a reproducible Docker–Gazebo–PX4 SITL data capture pipeline; eight physics-based heuristic labelling rules; a sliding-window ML framework with explicit train/test isolation; and a graduated post-detection mitigation protocol. Future work will focus on physical hardware validation, gradual-attack sensitivity analysis, and integration of the GTI into the PX4 EKF2 framework.